

CoBit / BitstreamDiffusion — Progress Report

Friday 4 – Wednesday 9 September 2026

Status against the original task list

Task	Status	Headline
1. Reproduce best baselines	DONE	replicated exactly; gap was churn alone
2. CFG + other guidance mechanisms, calibrate	DONE	CFG best, +0.081; study CLOSED
3. Nature of errors on TinyGSM	DONE	guidance harms code validity, not reasoning
4a. Temporal ordering — at inference	DONE	no benefit; random is harmful
4b. Temporal ordering — trained (L2R/R2L/random)	NOT DONE	inconclusive: control arm degenerate
5. Logits: softmax vs score matching	DONE	softmax $\equiv$ BCE (proved); SM vs CE negative

The most consequential finding of the week was not on the task list. Every training run in this environment diverged before $\sim 12k$ steps, which blocked all training experiments for four days. It is now solved: the cause is the **learning rate**, not the software environment. At $1r\ 3e-5$ a from-scratch run reached 20,000 steps cleanly where $3e-4$ diverges 8/8 times.

1 Task 1 — Reproduce best baselines **DONE**

Forensic replication audit against the collaborator's result: **verdict A, exact replication**. Our 0.1385 and their ~ 0.29 differ by **one factor: EDM stochastic churn**, worth +0.1622 and free in compute. Two churn code paths existed (`--gamma` vs `--churn_gamma`); the studies used different ones.

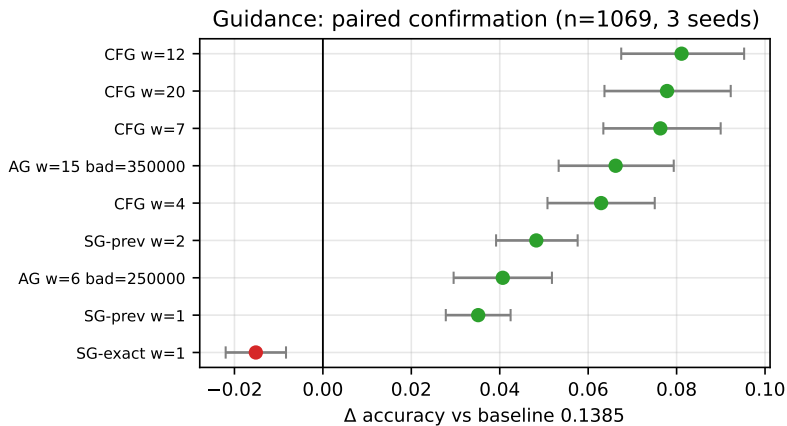
Baseline hierarchy on the 1319-problem GSM8K test set:

Setting	Accuracy	Note
Regime A (DDIM, $\gamma=0$, 500 steps)	0.1385	original arm
Regime B (FKC-EM, churn 0.41, 1024 steps)	~ 0.29	canonical, collaborator-matched
Production 500k, karras/DDIM, 256 steps	0.1640	anchor used throughout

Both regimes are retained and **every result is labelled with its regime**; they are never pooled.

2 Task 2 — Guidance mechanisms **DONE** (CLOSED)

Three mechanisms implemented and compared: classifier-free (CFG), AutoGuidance (AG), and Self-Guidance (SG-prev, SG-exact). Confirmation runs use 1069 problems $\times$ 3 seeds, paired bootstrap.



Config	Accuracy	Δ	95% CI	p	NFE
CFG $w=12$	0.2196	+0.0811	[+0.068, +0.095]	$< 10^{-5}$	1024
CFG $w=7$	0.2148	+0.0763	[+0.063, +0.090]	$< 10^{-5}$	1024
AG $w=15$ (bad@350k)	0.2047	+0.0662	[+0.053, +0.079]	$< 10^{-5}$	1024
SG-prev $w=2$	0.1868	+0.0483	[+0.039, +0.058]	$< 10^{-5}$	512
SG-exact $w=1$	0.1233	-0.0152	[-0.022, -0.008]	$< 10^{-5}$	1024

Findings. CFG is the strongest mechanism and saturates around $w \approx 7-12$. SG-prev gives about 60% of CFG’s gain at **half the NFE**, so it is the compute-efficient option. SG-exact actively hurts. **Mechanism:** guidance is not a correction — it fixes $\sim 10\%$ of problems and breaks $\sim 20-25\%$ regardless of regime, so its net value is arithmetic and only positive above a break-even baseline accuracy of $\approx 33\%$.

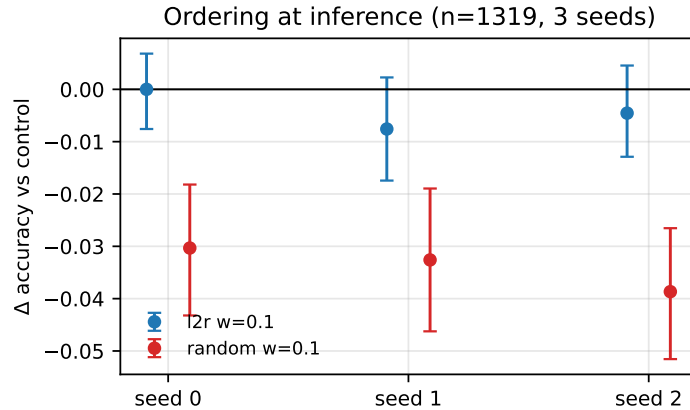
3 Task 3 — Nature of errors on TinyGSM **DONE**

Taxonomy: **T1** non-executing program, **T2** executes but wrong number, **T3** token-level degeneration.

The baseline’s dominant failure is T1 (98/250 = 39%) — the model emits programs that do not run. Under guidance, **T2 falls while T1 rises**: guidance damages *code validity*, not arithmetic reasoning. T3 appears only under strong guidance. Invalid-token rate is $3 \times 10^{-6} - 5 \times 10^{-5}$, i.e. bit $\rightarrow$ token decoding is not the problem.

4 Task 4a — Temporal ordering at inference **DONE**

Schedule: $t_j(t) = \text{clip}(t(1+w) - w u_j, 0, 1)$, with u_j a denoising *priority* over the generated **suffix only**; prompt keeps the global σ . Verified causally active: at $w=0.5$ the per-position σ spread is $192\times$, first suffix token cleanest; at $w=0$ it is bit-identical to the current model.



Arm	Accuracy	Δ	significant	Verdict
control ($w=0$)	0.1387	—	—	—
L2R $w=0.1$	0.1347	-0.0040	0/3 seeds	no effect
random $w=0.1$	0.1049	-0.0339	3/3 , $p < 10^{-4}$	harmful
L2R $w \geq 0.25$	0.0000	-0.176	3/3	collapse

Applying an ordering only at decoding does not help. The L2R/random asymmetry shows the intervention is real — direction matters — but nothing improves. Expected, since the model was *trained* with uniform σ : it tolerates a $\sim 2.9\times$ spread with no gain and breaks at $\sim 9\times$. 512 steps does not rescue it, so it is not a too-short-trajectory artefact.

5 Task 4b — Temporal ordering, **TRAINED NOT DONE** (inconclusive)

Implementation is complete and tested (24 tests): σ is drawn per example, mapped to normalised time by the *same* map the sampler uses, offset per token, mapped back and expanded blockwise to bits. All three modes provably produce *different* σ fields; L2R and R2L are mirror images; random is redrawn per example *and* per

step; $w=0$ leaves the gradient hash unchanged. Four arms (control / L2R / R2L / random, $w=0.25$) fine-tuned from the same production 500k checkpoint at $1r\ 3e-5$; none diverged.

The experiment is nonetheless inconclusive, because the control arm is degenerate. At the matched 5k checkpoint the uniform- σ control scores **0.0000** while the ordered arms score 0.07–0.12. Taken at face value that reads as “ordering massively improves TinyGSM”. It is not credible and is not claimed.

Diagnosis. The control’s weights moved only 4.65% from init and its EMA is healthy, so the model is not destroyed. The likely cause is the σ -sampling distribution: production converged under its entropy-adapted draw, but a fresh fine-tune restarts the entropy warmup (`entropy_warmup_steps=40000`) and therefore trains under the base log-normal draw. Every arm suffers that mismatch; the control, whose σ field is pure and unspread, appears to suffer it most. Verified under two independent sampling schedules (entropic and karras), so it is not a decoder artefact.

What is needed: a control that provably retains the init’s accuracy — either by preserving the entropy schedule across the fine-tune, or by training all arms from scratch under a common schedule. Not attempted within this window.

6 Task 5 — Logits: softmax vs score matching **DONE**

Softmax is not a distinct objective, and this is proved rather than assumed. The model already emits **one logit per bit** (`out_dim=1`, `vocab_size=2`). A 2-class softmax is shift-invariant, so it has a single degree of freedom $d = z_1 - z_0$ and

$$\text{softmax}(z_0, z_1)_1 = \sigma(d), \quad \text{CE}(z, y) = \text{BCEWithLogits}(d, y)$$

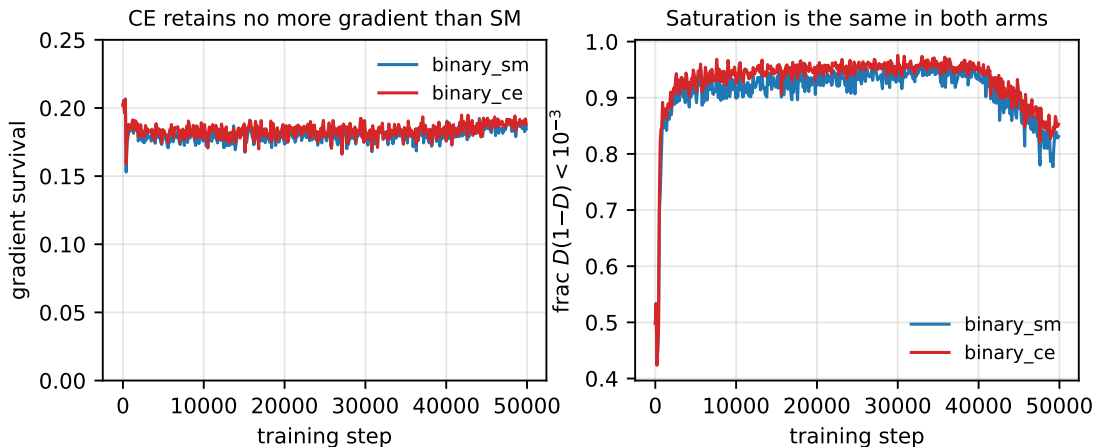
exactly; the 2-class head merely splits the same gradient across two logits ($\partial L / \partial z_1 = -\partial L / \partial z_0 = \partial L / \partial d$). Verified to 10^{-12} in 5 tests. **No GPU was spent on it.**

binary_sm vs binary_ce — definitive, from scratch, and negative

Both arms trained **from scratch for 50,000 steps** at $1r\ 3e-5$, **neither diverged**, differing in exactly four keys (`train.loss_type` plus three output paths). Evaluated on 1319 problems $\times$ 3 seeds, karras/DDIM, identical settings.

Arm	Mean accuracy	Correct / 1319	Stability
binary_sm	0.00027	0.4	stable to 50k
binary_ce	0.00100	1.3	stable to 50k
Δ	+0.00073	one problem	—

Neither objective produces measurable task performance at this budget, and a one-problem difference is not evidence of anything. The gradient diagnostics — computed identically in both arms and therefore directly comparable — agree:

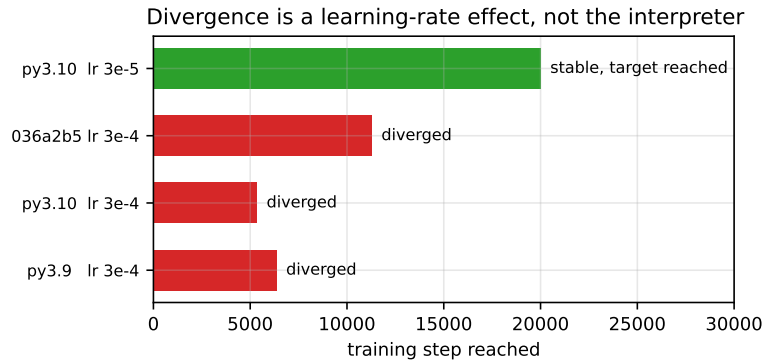


	SM @1k	CE @1k	SM @25k	CE @25k
gradient survival	0.1854	0.1866	0.1805	0.1808
saturation $D(1-D) < 10^{-3}$	0.846	0.865	0.942	0.956
logit mean	17.9	20.1	25.1	29.4

CE’s theoretical advantage does not appear. The motivation for CE was that its gradient lacks the $D(1-D)$ factor that suppresses SM’s. In a from-scratch matched pair at 50k, gradient survival is identical to three decimals. **This is the cleanest test the objective branch has had, and it is negative.**

Caveat, stated deliberately: $1r\ 3e-5$ establishes a **stable** regime, not a **tuned** one. These arms are compared only to each other, never to production’s $3e-4$ trajectory. A comparison at a tuned learning rate and a convergent budget remains untested.

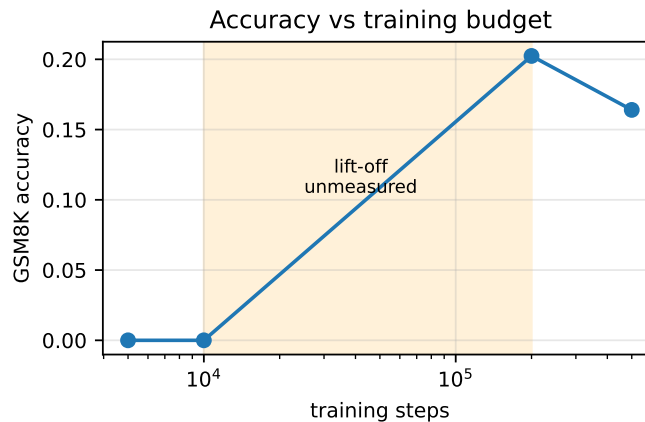
7 The infrastructural finding (not on the task list)



Eight from-scratch runs diverged before $\sim 12k$ steps at $1r\ 3e-4$, **including the production-era commit itself**. Ruled out along the way: corpus (σ_{data} fingerprint matches to ~ 8 s.f.), data config (identical on 22 keys), our post-June code (fp32 forward+loss+backward **bit-identical** to the production commit), and the Python interpreter — a 3.10 rebuild diverged at 5,346, falsifying a hypothesis I had put weight on.

At $1r\ 3e-5$ a from-scratch run reached 20,000 steps cleanly, as did all four ordering arms. The learning rate is the identified factor. *Caveat:* $3e-5$ establishes a **stable** regime, not a **tuned** one. It is $10\times$ below production’s value, so absolute accuracies in this regime are compared only against the matched control and never against production’s $3e-4$ trajectory.

8 Why accuracy could not be measured on short runs



GSM8K accuracy is **exactly 0.0000 at both 5k and 10k training steps** (measured, both objectives), and the first non-zero point we have is 0.2024 at 200k. Combined with divergence before 12k, the trainable range and the range where accuracy exists **did not overlap** — which is why the objective branch was blocked rather than merely slow.

9 Currently running

Only the L2R 30k retrain (its checkpoint was corrupted by a disk-full event) and the control’s matched 5k evaluation. Neither changes any verdict above.

10 Not done

1. **A valid ordering-as-training verdict.** Blocked on a control arm that does not retain its initialisation’s accuracy. The implementation is finished and tested; only the experimental setup is at fault.
2. **CE vs SM at a convergent budget.** The 50k result is clean and negative, but at `1r 3e-5` neither arm is near convergence. Whether CE helps a *converged* model is untested.
3. **Accuracy lift-off point** between 10k and 200k steps — unmeasured, and it sets the cost of every future training comparison.
4. **Why production tolerated `1r 3e-4` for 500k steps.** Their working tree is permission-denied to us, so this stays open.
5. **Ordering at other strengths, and R2L at inference** — deliberately not run; the first results did not justify the spend.

11 Honest notes

Three self-inflicted failures cost ~ 15 GPU-h and a night of wall-clock: an over-strict guidance guard failed 9/9 evaluation cells; the ordering control returned probabilities from σ_{N-1} instead of a final denoise, scoring 0.000 against a true 0.164; and a disk-full event killed four training arms mid-checkpoint. Each was caught before it entered a result. The second was caught *only* because a production anchor was included in the evaluation — without it, seven cells of confidently wrong nulls would have been reported.

A fourth was caught this morning: the entropic sampling schedule is fitted per-run, so each fine-tuned arm had refitted its own decoder schedule ($\max |\text{cdf} - \text{prod}|$ up to 0.545). Evaluating that way compares arms differing in both training *and* decoding. All evaluations now use the analytic karras schedule, identical for every arm by construction.